



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Router chains e o conceito de grafo de estado

Aula 12/14 — Módulo 4: LangGraph e Encerramento



Prof. Jorge Luiz Gomes



1h40min



Router Chain · StateGraph mental · LangGraph motivação



Andaime 50%

Resultados do check-in (Aula 11) + agenda de hoje

📊 CHECK-IN ANÔNIMO — AULA 11

[Professor lê aqui os resultados reais do formulário antes da aula]



Conceito que precisa de mais prática: resposta mais frequente será lida aqui — ajustar o ritmo da aula se necessário



O que faltou nos materiais: ajuste fino de conteúdo para as Aulas 13–14 com base na resposta

AGENDA DA AULA 12

1

Por que chains lineares não bastam — o problema dos branches

12 min

2

Router Chain — classificar intenção e rotear para handlers

15 min

3

Grafo de estado — modelo mental, sem código ainda

13 min

4

Por que LangGraph — AgentExecutor vs. StateGraph

10 min

5

🚀 Demo — Router Chain 3 rotas ao vivo

20 min

6

🖥️ Lab — router para o domínio do grupo (50% lacunas)

30 min

Glossário

Router Chain

Chain que primeiro classifica a intenção da entrada e depois despacha para uma sub-chain (handler) especializada, em vez de usar um único prompt genérico para tudo.

Handler

Chain dedicada a uma única rota ou intenção, com prompt e lógica otimizados para aquele caso específico, em vez de um prompt genérico tentando cobrir todos os cenários.

with_structured_output()

Método do LangChain que força o LLM a retornar dados num schema Pydantic definido, eliminando parsing manual de JSON e reduzindo erro de formatação na saída.

Node (grafo de estado)

Unidade de processamento do grafo: recebe o estado atual, executa uma função e retorna a atualização do estado — equivalente a uma etapa de chain, mas com posição explícita no fluxo.

Classificador de intenção

Componente — geralmente um LLM com structured output — que decide a categoria da mensagem do usuário antes de qualquer processamento; a saída dele determina qual handler roda.

Grafo de estado (state graph)

Modelo mental onde cada etapa do pipeline é um node, as transições entre etapas são edges, e um objeto de estado compartilhado flui e é atualizado a cada node visitado.

Conditional edge

Aresta de um grafo de estado cujo destino é decidido em tempo de execução por uma função de roteamento, em vez de ser fixo — é o que implementa a lógica do Router como grafo.

AgentExecutor vs. Router Chain vs. LangGraph

Três níveis de controle de fluxo: AgentExecutor deixa o LLM decidir tudo a cada turno; Router Chain fixa duas rotas (classificar → despachar); LangGraph explicita todo o grafo como código auditável.

01

Por que chains lineares não bastam

Toda chain que você construiu até hoje executa em linha reta: $A \rightarrow B \rightarrow C$.

Quando o fluxo precisa decidir entre caminhos diferentes com base no input, você precisa de branches.

O limite das chains lineares

Da Aula 01 até a Aula 08, toda chain executou em sequência fixa. Isso funciona perfeitamente para um único tipo de tarefa — mas no mundo real, o usuário pode perguntar qualquer coisa:

✅ CHAIN LINEAR — FUNCIONA

Quando **toda pergunta** vai para o mesmo fluxo:

pergunta → RAG → prompt → LLM → resposta

Funciona para um assistente de documentos que **SÓ** responde sobre documentos. Simples, previsível, rápido.

❌ CHAIN LINEAR — QUEBRA

Quando perguntas diferentes precisam de fluxos diferentes:

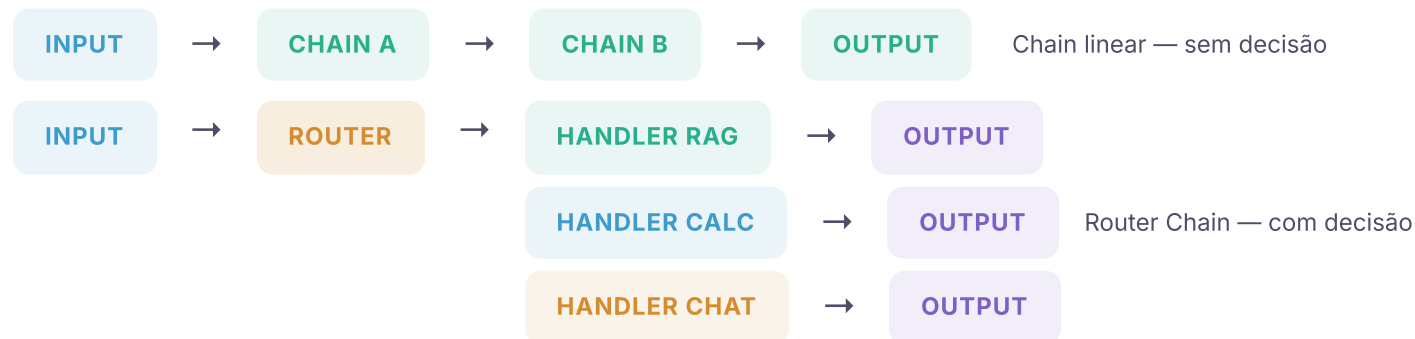
"Qual a cláusula 5?" → precisa do **RAG**

"Quanto é 450×1.12 ?" → precisa da **calculadora**

"Oi, tudo bem?" → precisa de **conversa**

Mandar tudo para o RAG é desperdício — e às vezes responde errado.

COMPARAÇÃO DE ARQUITETURAS



02

Router Chain

Um classificador de intenção que decide qual handler executar. O classificador é um LLM com structured output — retorna a rota como string, não texto livre.

Classificador de intenção — o coração do Router

O classificador é uma chain LCEL simples que recebe o input do usuário e retorna **uma palavra** indicando a rota correta. Usar Literal do Pydantic garante que o LLM retorne apenas opções válidas:

CLASSIFICADOR_INTENCAO.PY

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from pydantic import BaseModel
from typing import Literal

llm = ChatOllama(model="qwen3.6:27b", temperature=0)

# Schema Pydantic para a saída do classificador
class Rota(BaseModel):
    destino: Literal["rag", "calculadora", "conversa"]
    # Literal garante que o LLM retorne APENAS uma dessas 3 strings
    # Se retornar algo diferente → ValidationError → fallback para "conversa"

prompt_classificador = ChatPromptTemplate.from_messages([
    ("system", """"Você é um roteador de intenções. Classifique o input do usuário em exatamente um destino:
    - rag: perguntas sobre documentos do domínio, manuais, contratos, regulamentos
    - calculadora: cálculos matemáticos, conversões numéricas, porcentagens
    - conversa: saudações, perguntas gerais, agradecimentos, qualquer outra coisa

    Retorne apenas o JSON com o campo 'destino'. """"),
    ("human", "{input}"),
])

# Chain do classificador – with_structured_output garante Pydantic válido
chain_classificador = prompt_classificador | llm.with_structured_output(Rota)

# Testar o classificador isoladamente
for inp in ["Qual a cláusula 5 do contrato?", "Quanto é 450 × 0.88?", "Oi tudo bem?"]:
    rota = chain_classificador.invoke({"input": inp})
    print(f'"{inp[:30]}..." → {rota.destino}')
# 'Qual a cláusula 5 do contrato?...' → rag
# 'Quanto é 450 × 0.88?...' → calculadora
# 'Oi tudo bem?...' → conversa
```

 FUNDAMENTAÇÃO — O CUSTO DE LATÊNCIA DE UM ROUTER BASEADO EM LLM

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) chama atenção para um custo que o classificador acima introduz silenciosamente: cada chamada de roteamento via LLM soma de 500ms a 2 segundos de latência antes mesmo de a rota escolhida começar a trabalhar — em um chat interativo, isso é atraso perceptível somado em cima da resposta final. O autor recomenda começar exatamente como esta aula faz, com um classificador LLM, para validar a lógica de roteamento; se a latência virar problema e as rotas forem bem distintas entre si, a alternativa é treinar um classificador leve sobre embeddings da pergunta (10-50ms, uma fração do custo). Vale usar roteamento quando existem fontes de dados claramente distintas e o custo de buscar na fonte errada supera o overhead do roteamento — exatamente o caso de rag vs. calculadora vs. conversa desta aula.

Handlers — uma chain por rota

Cada rota do Router corresponde a uma chain especializada. A chain correta é executada **apenas quando o classificador a selecionar** — sem overhead das outras:

HANDLERS.PY

```
from langchain_core.runnables import RunnableLambda

# Handler 1 – RAG (reutiliza o retriever e prompt do CKP02)
prompt_rag = ChatPromptTemplate.from_template(
    "Use o contexto abaixo para responder a pergunta. Se não souber, diga que não sabe.\n"
    "{contexto}\n\nPergunta: {input}"
)
handler_rag = (
    {"contexto": retriever | RunnableLambda(lambda docs: "\n\n".join(d.page_content for d in docs)),
     "input": lambda x: x["input"]}
    | prompt_rag | llm | StrOutputParser()
)

# Handler 2 – calculadora (sem LLM para a matemática)
def _calcular(dados: dict) -> str:
    # Primeiro extrai a expressão Python via LLM, depois calcula
    expressao = (ChatPromptTemplate.from_template(
        "Extraia apenas a expressão Python do cálculo pedido. Retorne somente a expressão, sem texto: {input}"
    ) | llm | StrOutputParser()).invoke(dados)
    try:
        resultado = eval(expressao.strip(), {"__builtins__": {}}, {})
        return f"Resultado: {resultado} (expressão: {expressao.strip()})"
    except:
        return "Não foi possível calcular. Tente escrever a expressão de outra forma."
handler_calculadora = RunnableLambda(_calcular)

# Handler 3 – conversa genérica
handler_conversa = (
    ChatPromptTemplate.from_messages([
        ("system", "Você é um assistente simpático do domínio do grupo. Responda de forma amigável e concisa."),
        ("human", "{input}"),
    ]) | llm | StrOutputParser()
)

# Mapa de handlers por destino
HANDLERS = {"rag": handler_rag, "calculadora": handler_calculadora, "conversa": handler_conversa}
```


Montando o Router — classificar → despachar

ROUTER_CHAIN.PY

```
from langchain_core.runnables import RunnableLambda

def rotear(dados: dict) -> str:
    """Classifica a intenção e despacha para o handler correto."""
    inp = dados["input"]
    rota = chain_classificador.invoke({"input": inp})

    print(f"[ROUTER] '{inp[:40]}' → {rota.destino}")

    # Despachar para o handler correspondente
    handler = HANDLERS.get(rota.destino, HANDLERS["conversa"]) # fallback: conversa
    return handler.invoke({"input": inp})

# Router chain final — RunnableLambda torna a função uma Runnable LCEL
router_chain = RunnableLambda(rotear)

# Testar o sistema completo
testes = [
    "Qual o prazo de garantia no contrato?", # → rag
    "Quanto é 1.250 com 15% de desconto?", # → calculadora
    "Você pode me ajudar com algo hoje?", # → conversa
    "Qual a multa por rescisão antecipada?", # → rag
]

for t in testes:
    print(f"\n{'='*50}\nInput: {t}\nOutput: {router_chain.invoke({'input': t})}")
```

💡 FALLBACK OBRIGATÓRIO NO GET()

Sempre use `HANDLERS.get(destino, HANDLERS["conversa"])` — nunca `HANDLERS[destino]` diretamente. Se o LLM retornar um destino inesperado (raro com structured_output, mas possível), o sistema cai no handler de conversa em vez de levantar KeyError em produção.

03

Grafo de estado (modelo mental)

Antes de escrever código LangGraph na Aula 13, construir o modelo mental: o que é um grafo de estado, quais problemas ele resolve que o Router e o AgentExecutor não resolvem.

Grafo de estado — o modelo mental

Um grafo de estado tem 4 elementos fundamentais. Todos os sistemas que você já conhece são casos especiais deste modelo:



Estado (State)

Um dicionário compartilhado que todos os nós do grafo podem ler e escrever. É a "memória de trabalho" de toda a execução. Ex: `{"mensagens": [...], "rota": None, "resposta": None}`. O estado é passado de nó em nó — cada nó recebe o estado atual e retorna as mudanças.



Nós (Nodes)

Funções Python que recebem o estado e retornam um dict com as atualizações. Ex: um nó "classificar" lê `state["input"]` e escreve `{"rota": "rag"}`. O grafo faz merge automaticamente — o nó só precisa retornar o que mudou.



Arestas (Edges)

Conexões entre nós. Podem ser fixas (`nó_A → nó_B` sempre) ou condicionais (`nó_A → nó_B` ou `nó_C` dependendo do estado). Arestas condicionais são o superpoder do LangGraph — é onde a decisão fica explícita no grafo, não embutida na chain.



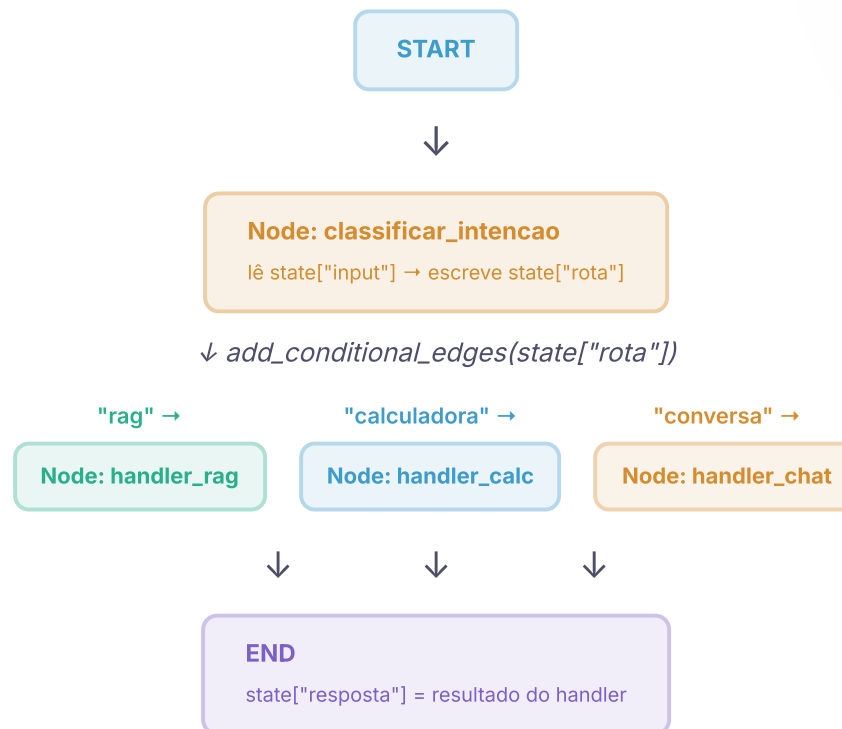
Nós especiais — START e END

START injeta o input inicial no estado. END finaliza a execução e retorna o estado final. O grafo não termina até que um nó aponte para END.

O Router desta aula como grafo de estado

A Router Chain que você vai construir no lab pode ser modelada como um grafo. Isso prepara o modelo mental para a Aula 13:

ROUTER CHAIN COMO STATEGRAPH — SEM CÓDIGO, SÓ O DIAGRAMA



NA AULA 13 VOCÊ VAI IMPLEMENTAR EXATAMENTE ESTE GRAFO EM LANGGRAPH

Com `StateGraph`, `add_node()`, `add_conditional_edges()` e `compile()`. O diagrama mental desta aula vira código Python. Guardar este diagrama — ele será a referência na implementação.

04

Por que LangGraph

AgentExecutor (M3) para loops simples. Router Chain (hoje) para branching manual. LangGraph para quando o fluxo tem branches condicionais, loops com saída, paralelismo e revisão humana.

AgentExecutor vs. Router Chain vs. LangGraph

CARACTERÍSTICA	AGENTEXECUTOR (M3)	ROUTER CHAIN (HOJE)	LANGGRAPH (A13)
Fluxo	Loop linear: T→A→O→repeat	Branching: 1 input → 1 de N handlers	Grafo: qualquer topologia — ciclos, paralelos, convergências
Decisão	Implícita: o LLM decide no Thought	Explícita: classificador → get(destino)	Explícita: add_conditional_edges() visível no grafo
Estado	intermediate_steps (opaco)	Sem estado compartilhado	TypedDict persistente, acessível em qualquer nó
Checkpointing	❌ Não tem	❌ Não tem	✅ MemorySaver — pausar e retomar
HITL (revisão humana)	❌ Não suporta	❌ Não suporta	✅ interrupt_before/after — o humano aprova antes de continuar
Paralelismo	❌ Sequencial	❌ 1 handler por vez	✅ Nós paralelos com Send()
Quando usar	Tools dinâmicas, escopo aberto	Roteamento simples, poucas rotas	Pipelines complexos, produção, auditoria

⚠️ NÃO SUBSTITUIR — EMPILHAR

LangGraph não substitui AgentExecutor ou Router Chain — cada um resolve um problema diferente. Um agente LangGraph pode ter um nó que internamente usa um AgentExecutor para tool calls. Escolha a ferramenta certa para cada camada do sistema.

06

Lab — Notebook Aluno (30 min)

50% de lacunas — o aluno escreve o prompt do classificador, o handler de conversa e a função de despacho. Depois adapta para o domínio do grupo.



```
!pip install langchain langchain-ollama langchain-chroma pydantic -q

from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
from pydantic import BaseModel
from typing import Literal
import os
from google.colab import userdata

os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")
llm = ChatOllama(model="qwen3.6:27b", temperature=0)
embeddings = OllamaEmbeddings(model="nomic-embed-text")
retriever = Chroma(persist_directory="/content/ckp02",
                    embedding_function=embeddings).as_retriever(search_kwargs={"k":3})

# 🚩 LACUNA 1: Rota com Literal – adicionar os destinos do domínio do grupo
class Rota(BaseModel):
    destino: Literal[_____, _____, _____] # ex: "rag", "calculadora", "conversa"

# 🚩 LACUNA 2: prompt do classificador – descrever cada destino do domínio
prompt_clf = ChatPromptTemplate.from_messages([
    ("system", _____), # descrever cada rota com exemplos do domínio
    ("human", "{input}"),
])
chain_clf = prompt_clf | llm.with_structured_output(Rota)

# Handler RAG (pronto)
handler_rag = ({ "contexto": retriever | RunnableLambda(lambda d: "\n".join(x.page_content for x in d)),
                 "input": lambda x: x["input"]}
               | ChatPromptTemplate.from_template("Contexto:\n{contexto}\n\nPergunta: {input}")
               | llm | StrOutputParser())

# 🚩 LACUNA 3: implementar handler_conversa (chain simples com system prompt amigável)
handler_conversa = (
    ChatPromptTemplate.from_messages([
        ("system", _____), # persona do assistente do domínio do grupo
        ("human", "{input}"),
    ]) | llm | StrOutputParser()
)

HANDLERS = {"rag": handler_rag, "calculadora": handler_calculadora, "conversa": handler_conversa}

# 🚩 LACUNA 4: implementar a função rotear(dados) e montar router_chain
def rotear(dados: dict) -> str:
    rota = chain_clf.invoke({"input": dados[_____]})
    print(f"[ROUTER] {rota.destino}")
    return HANDLERS.get(_____, HANDLERS["conversa"]).invoke(dados)

router_chain = RunnableLambda(_____)
```


with_structured_output() — structured output revisitado

Já vimos `with_structured_output()` na Aula 03 com Pydantic v2. No classificador, ele tem um papel crítico de segurança:

❌ SEM PYDANTIC/LITERAL — FRÁGIL

```
# LLM pode retornar qualquer texto
"rag"          # ✅ ok
"RAG"          # ❌ KeyError no dict
"use o rag"    # ❌ KeyError no dict
"Vou usar rag" # ❌ quebra em produção
```

✅ COM LITERAL — ROBUSTO

```
# Literal garante exatamente 1 de N
class Rota(BaseModel):
    destino: Literal["rag","calc","chat"]
# O LLM é forçado (via JSON schema)
# a retornar APENAS um dos 3 valores
# "RAG" → ValidationError → tratável
```

🔗 CONEXÃO COM AULA 03 — PYDANTIC V2 E STRUCTURED OUTPUT

Na Aula 03 usamos Pydantic para garantir que a extração de dados retornasse campos tipados. Aqui usamos o mesmo padrão para garantir que o classificador retorne sempre um destino válido. Pydantic não é só para extração — é o mecanismo de contrato entre o LLM e o código Python em qualquer contexto.

Síntese da Aula 12



Chains lineares têm limite. Quando perguntas diferentes precisam de fluxos diferentes, é necessário branching. O Router Chain resolve isso sem LangGraph para casos simples.



Router = classificador + handlers + despacho. O classificador usa `Literal + with_structured_output()` para garantir retorno válido. Fallback no `get()` evita `KeyError` em produção.



Grafo de estado = State + Nodes + Edges + START/END. O Router desta aula já é um grafo — falta apenas a formalização em LangGraph (Aula 13).



LangGraph resolve o que Router e AgentExecutor não resolvem: estado persistente, checkpointing, revisão humana (HITL) e paralelismo.



Aula 13: implementar este grafo em código com `StateGraph`, `add_node()`, `add_conditional_edges()`, `MemorySaver` e HITL. O diagrama desta aula vira código Python.

Perguntas frequentes

PERGUNTA	RESPOSTA
"Router Chain ou AgentExecutor — qual a diferença real na prática?"	No AgentExecutor, o LLM decide a cada turno qual tool usar — decisão implícita no Thought. No Router Chain, o LLM classifica a intenção uma vez e o código despacha — decisão explícita e rastreável. Para logs de auditoria ("por que o sistema tomou essa decisão?"), o Router é mais fácil de explicar. Para tarefas que precisam de múltiplas tools em sequência imprevisível, o AgentExecutor é mais flexível.
"Posso ter mais de 3 rotas no Router?"	Sim — basta adicionar mais valores no Literal e mais handlers no dict. Na prática, acima de 5–6 rotas o prompt do classificador fica complexo e a acurácia cai. Se você precisar de muitas rotas, considere hierarquia: um router principal que despacha para sub-routers mais especializados. Ou migrar para LangGraph com conditional_edges — mais robusto para grafos complexos.
"O classificador pode errar a rota?"	Sim — e por isso o fallback para "conversa" existe. Com temperature=0 e Literal, o erro é raro mas possível para perguntas ambíguas. Monitorar a distribuição de rotas em produção (logar rota.destino) para detectar erros sistemáticos. Se a rota "rag" estiver sendo escolhida para cálculos, o prompt do classificador precisa de exemplos negativos ("NÃO use para cálculos matemáticos").

Python novo desta aula

CONCEITOS_PYTHON_AULA12_2SEM.PY

```
# 1. Literal – tipo que aceita apenas valores específicos
from typing import Literal

def fn(cor: Literal["vermelho", "azul", "verde"]): ...
# fn("amarelo") → erro de type checking (mas não RuntimeError em Python puro)
# com Pydantic: Literal em BaseModel → ValidationError em runtime

# 2. with_structured_output(PydanticModel) – reforçar Literal em runtime
from pydantic import BaseModel

class Rota(BaseModel):
    destino: Literal["a", "b"]

chain = prompt | llm.with_structured_output(Rota)
result = chain.invoke({"input": "..."})
result.destino # → sempre "a" ou "b" – nunca outro valor

# 3. dict.get(key, default) – fallback seguro
d = {"a": 1, "b": 2}
d.get("c", "default") # → "default" (sem KeyError)
d["c"]                 # → KeyError (quebra em produção)

# 4. RunnableLambda – transformar qualquer função em Runnable LCEL
from langchain_core.runnables import RunnableLambda

def minha_fn(dados: dict) -> str: return dados["x"].upper()
runnable = RunnableLambda(minha_fn)
runnable.invoke({"x": "hello"}) # → "HELLO"
# RunnableLambda é o "adaptador" que coloca qualquer função no pipe LCEL

# 5. Lambda em dicts de chain – LCEL fan-out
chain = {
    "contexto": retriever | RunnableLambda(lambda docs: "\n".join(d.page_content for d in docs)),
    "input": lambda x: x["input"],
} | prompt | llm # dict com lambdas = fan-out LCEL (executa em paralelo)
```

Router patterns em produção

PLATAFORMAS DE ATENDIMENTO COM IA

Triagem de tickets

O ticket do cliente chega e um classificador roteia para: resposta automática (FAQ), agente humano nível 1, ou escalation técnico. Exatamente o padrão Router desta aula — o classificador usa embeddings + LLM para decidir a rota.

APIS MULTIMODAIS

Router de modalidade

A API detecta se o input é texto, imagem, código ou áudio e roteia para o modelo especializado correto: um modelo de texto, um modelo de transcrição de áudio, um modelo de geração de imagem. Um router interno em grande escala — mesmo conceito desta aula.

PLATAFORMAS NO-CODE DE AUTOMAÇÃO

Nó de decisão

Plataformas no-code de automação têm nó de decisão que implementa exatamente o Router: uma expressão ou LLM decide por qual branch o dado vai seguir. Em workflows de automação, o padrão de routing é um dos mais usados — triagem de emails, classificação de documentos, escalation de alertas.

Onde estamos no semestre

M1-M3 ✓ LCEL · RAG · Agentes · CKP01 + CKP02 + CKP03 entregues

A12 ✓ Router Chain · Classificador de intenção · Grafo de estado (modelo mental)

A13 → **LangGraph:** StateGraph com TypedDict · add_node() · add_edge() · add_conditional_edges() · MemorySaver · HITL

A14 → **Encerramento:** Tendências 2026 · próximos passos na carreira · retrospectiva do semestre

PREPARAR PARA A AULA 13

Desenhar (em papel ou markdown) o grafo do Router desta aula com os 4 elementos: State TypedDict com os campos, 4 nodes (classificar + 3 handlers), conditional edge saindo do classificador, END. Esse diagrama será a base do código LangGraph da Aula 13.

Preview — Aula 13: LangGraph em código

O QUE VOCÊ VAI IMPLEMENTAR

```
from langgraph.graph import StateGraph, START, END
```

- **TypedDict** como schema do estado compartilhado
- **add_node()** para cada função de processamento
- **add_edge()** para conexões fixas
- **add_conditional_edges()** para o router
- **graph.compile(checkpointer=MemorySaver())**
- **interrupt_before** para HITL

A DIFERENÇA QUE VAI SURPREENDER

Com MemorySaver, o grafo pode ser **pausado** no meio da execução — o estado é salvo. Quando o usuário aprova (HITL), a execução continua do ponto exato onde parou. Isso é impossível com AgentExecutor e Router Chain.

Caso de uso real: gerador de relatório que precisa de aprovação humana antes de enviar por email. O grafo pausa antes do nó "enviar_email", o humano aprova no Gradio, o grafo continua.

Guia rápido — qual arquitetura usar?

FLUXOGRAMA DE DECISÃO

Tarefa única, fluxo fixo?

→ **Chain LCEL** — mais simples, mais rápido, mais barato (M1–M2)

Múltiplos tipos de input, rotas fixas e conhecidas?

→ **Router Chain** — decisão explícita, auditável, sem overhead de loop (esta aula)

Ferramentas dinâmicas, sequência imprevisível?

→ **AgentExecutor** — flexibilidade máxima para tool use dinâmico (M3)

Ciclos, paralelismo, HITL ou checkpointing?

→ **LangGraph** — controle total sobre o grafo de execução (A13)

05

Demo ao vivo (20 min)

Router Chain com 3 rotas — classificador, handlers e função de despacho. O professor mostra o roteamento sendo impresso a cada invocação.

Demo Prática ao Vivo — Router Chain ao vivo

O professor constrói ao vivo e executa os 4 testes de roteamento. Alunos acompanham o log `[ROUTER] '...' → destino :`

INPUT DO USUÁRIO	ROTA ESPERADA	HANDLER ATIVADO	POR QUE
"Qual a cláusula 5 do contrato?"	rag	handler_rag → retriever → LLM	Palavra "cláusula" + "contrato" → domínio de documentos
"Quanto é 1.250 com 15% de desconto?"	calculadora	handler_calculadora → extrai expressão → eval	Padrão numérico + palavra "desconto" → cálculo
"Oi, você pode me ajudar com algo?"	conversa	handler_conversa → LLM amigável	Saudação sem intenção específica → conversa
"Qual a multa por rescisão? E quanto seria em reais se o contrato é de R\$5.000?"	rag (1ª resposta)	handler_rag → retriever → LLM	Multi-parte: a pergunta dominante é sobre o contrato. O handler RAG responde sobre a cláusula; a calculadora não é ativada (limitação do Router simples — resolvida no LangGraph)

A LIMITAÇÃO QUE MOTIVA O LANGGRAPH

O último exemplo expõe a limitação do Router: perguntas multi-parte (RAG + cálculo) precisam de 2 rotas — mas o Router só escolhe 1. Em LangGraph, a aresta condicional pode ativar *dois nós* em paralelo ou sequência. Essa é a motivação concreta para a Aula 13.

Router Chain com 3 rotas para o domínio do grupo ★★★

Grupo 3-4 · 30 minutos · Google Colab

1. **Complete as 4 lacunas** — Literal com destinos reais do domínio, prompt do classificador descrevendo cada rota com exemplos específicos do domínio, persona do handler_conversa, função rotear completa.
2. **Teste os 3 cenários** — RAG (pergunta sobre documentos do grupo), calculadora (um cálculo relevante para o domínio) e conversa (saudação ou pergunta fora do escopo).
3. **Analise o roteamento** — troque uma palavra-chave na pergunta que vai para RAG para ver se o classificador ainda acerta. Ex: "documentação" → "manual" → "arquivo" — qual para de rotear corretamente?

Gabarito das lacunas

L1: Literal["rag", "calculadora", "conversa"] (ou os 3 destinos do domínio)
L2: System prompt descrevendo cada destino com exemplos do domínio real
L3: system="Você é um assistente de [DOMÍNIO]. Responda de forma amigável..."
L4: dados["input"] ; rota.destino ; rotear



Desafio — desenhar o grafo desta aula como StateGraph: em papel (ou célula markdown), mapear: quais seriam os nodes? Quais as edges? Qual a conditional edge? O estado TypedDict teria quais campos? Isso é o exercício de preparação para a Aula 13.

Referências

DOCS

LangChain — RunnableLambda e routing patterns. Como construir chains com branching usando RunnableLambda e with_structured_output.
python.langchain.com/docs/how_to/routing

DOCS

LangGraph — Conceitos de StateGraph, nodes e edges. A leitura recomendada antes da Aula 13. langchain-ai.github.io/langgraph/concepts/low_level/

BLOG

Anthropic Engineering — "Building Effective Agents" (2025). Seção sobre orchestrators e subagents — a motivação arquitetural para grafos de estado.
anthropic.com/engineering/building-effective-agents

LIVRO

Russell, S.; Norvig, P. — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 3 — Resolução de problemas como busca: a teoria por trás de grafos de estado em IA, base conceitual do LangGraph.

Branches explícitos

O Router Chain torna a decisão visível e rastreável. O modelo mental do grafo está pronto. Na Aula 13 isso vira código LangGraph com estado, checkpointing e revisão humana.

PRÓXIMA AULA — AULA 13

LangGraph — StateGraph, conditional edges e HITL

O diagrama desta aula vira código. Estado TypedDict, add_node(), add_conditional_edges(), MemorySaver, interrupt_before.

Preparar para a Aula 13

Desenhar o grafo do Router como StateGraph (State + 4 Nodes + conditional edge + END). Ler: langchain-ai.github.io/langgraph/concepts/low_level